

## Nexus AI - Project Interview Report

This document is a comprehensive guide to the Nexus AI Data Analyst project. It breaks down the system architecture, file structure, and specific functions used throughout the application. It is tailored to help you confidently explain the project's logic and workflows in an interview.

---

### 1. Project Overview

Nexus AI is a cutting-edge, AI-powered Data Analyst application designed for business users. It provides an intuitive interface for users to upload datasets (CSV), run automated analysis, view dynamic dashboard insights, chat with an AI assistant to query their data, and download detailed reports (PDF/Excel).

#### Tech Stack

- Frontend: Next.js (React 19), Tailwind CSS, Framer Motion (Animations), Recharts (Data Visualizations), Zustand (State Management), jsPDF (Report Generation)
- Backend: FastAPI (Python), Motor (Async MongoDB Driver)
- Database: MongoDB
- Styling: Custom "Neon Violet" / "Cyber AI" theme built with CSS variables.

---

### 2. Frontend Architecture (Next.js)

The frontend is a Single Page Application (SPA) structured to dynamically switch between different "views" or "steps" without reloading the page.

#### A. `src/app/page.tsx` (The Main Hub)

This is the central orchestration file. It manages the global UI layout, the sidebar navigation, and conditionally renders different views (Upload, Analyze, Dashboard, Chat, Reports).

Key Functions & Hooks:

- `setCurrentStep(step: Step)`: Controls which view the user is currently seeing (e.g., 'upload', 'dashboard', 'chat').
- `handleUploadSubmit()`: Triggered when the user clicks "Analyze Dataset". It simulates the uploading process, transitions the UI to the 'analyze' state, and kicks off the background agent workflow simulation.

# Nexus AI - Project Interview Report

- `handleSendMessage()`: Located in the Chat view.
- It takes the user's plain English query from the input box.
- Passes the query to the backend `/api/chat` endpoint (or utilizes client-side caching if the query was asked previously).
- Updates the `chatMessages` array to render the AI's response and any generated SQL code on the screen.
- `handleDownloadReport(type: 'pdf' | 'excel')`: Located in the Reports view.
- For PDF: Instantiates `jsPDF` and `jspdf-autotable` to dynamically draw a formatted, business-ready document summarizing dataset health and segment revenue.
- For Excel: Generates a well-structured CSV string and converts it to a Blob to force a browser download of the data.

## B. `src/components/InsightsCenter.tsx` (Dashboard Visuals)

This component is responsible for rendering the complex, dynamic dashboard view once data analysis is complete.

Key Structural Elements & Functions:

- **Recharts Integration:** Uses `ResponsiveContainer`, `BarChart`, `LineChart`, and `AreaChart` from the `recharts` library to render the visual representations of the data (e.g., Revenue Distribution, Growth Trends).
- **Dataset Health Report (Anomaly Detection):** Renders a business-friendly summary of the dataset's quality. Instead of showing complex Machine Learning standard deviations, it translates anomalies into a clear "Healthy/Warning/Critical" status.
- **Business Forecast Report:** Replaces complex AI terminology with simple predictive text (e.g., "Growing", "Stable", "Declining") to tell the user exactly what to expect in the next business period.

## C. `src/store/agentStore.ts` (Global State & Workflow Simulation)

This file uses Zustand (a lightweight state management library for React) to manage the multi-agent workflow system.

Key Functions & Logic:

- `simulateWorkflow(query: string)`: An asynchronous function that mocks the behavior of a swarm of AI agents working together.
- It iterates through different specialized "Agents" (e.g., Data Parsing, SQL Generation, Visualization).
- Uses `setTimeout` to mimic the processing delay of an LLM.
- Updates the state (`status: 'running' | 'completed'`) of each agent sequentially.
- `setActiveAgentId(id: string)`: Allows the user to click on a specific agent in the UI to view its detailed execution logs or specific output code.

## D. `src/components/AgentWorkflowBar.tsx` (Swarm Animation UI)

This component visually represents the data inside `agentStore.ts`.

# Nexus AI - Project Interview Report

Key Functions & Hooks:

- `useEffect`` (Auto-scroll): Listens to changes in the agent statuses and automatically scrolls the horizontal container to keep the currently "running" agent in the center of the user's screen.
- `map()`` rendering: Dynamically renders a pill/card for every agent in the workflow, glowing blue when active and green when completed.

## E. `src/app/login/page.tsx`` (Authentication)

Handles user login and signup.

Key Functions:

- `handleSubmit(e)``: Captures the user's email and password. Originally designed to POST to a backend `/api/auth`` endpoint, it was refactored for instant, client-side mockup access. It saves the `userEmail`` to browser `localStorage`` and redirects the user to the main dashboard (`/``) instantly.

---

## 3. Backend Architecture (FastAPI)

The backend is built with Python's FastAPI. Currently, it acts mostly as a mock endpoint provider to support the frontend UI.

### `backend/main.py``

This is the single entry point for the backend server.

Key Functions & Endpoints:

- `signup()` / `login()``: FastAPI `POST`` endpoints interacting with MongoDB (Motor client) to verify user credentials and check for existing accounts.
- `websocket_endpoint(websocket)``: Establishes a WebSocket connection.
- `mock_agent_stream(websocket)``: A function called by the websocket endpoint that loops through a list of stages (e.g., "Detecting data types", "Generating insights") and pushes these status updates to the frontend in real-time.
- `generate_analysis_for_df(df)``: A Pandas-based function designed to take a DataFrame, compute statistical metrics, find numeric and categorical columns, and format the data into a JSON structure that the frontend Recharts components can ingest.

---

## 4. Key Talking Points for Your Interview

# Nexus AI - Project Interview Report

If asked about your contribution or architectural decisions, highlight these points:

1. **User-Centric Design:** Emphasize how you translated complex AI outputs (like anomaly detection and forecasting) into business-friendly terminology. You removed ML jargon to make the dashboard actionable for executives.
2. **Instant Gratification & Caching:** Mention the `useRef` caching system built into the Chat view (`page.tsx`). If a user asks the same query twice, the frontend instantly returns the cached response rather than wasting time re-running the AI workflow.
3. **Advanced Frontend Features:** Discuss the native PDF generation using `jspdf`. Instead of relying on a complex, slow backend microservice for report generation, you implemented client-side PDF drawing which reduces server load and provides immediate file downloads for the user.
4. **Targeted AI Context:** You implemented a clickable "File Pill" system in the Chat interface. This allows users to specifically target which CSV files the AI should analyze, preventing the LLM from hallucinating or scanning irrelevant data, significantly improving prompt accuracy.